

Design and Implementation of a Simple Client-Server Application

Student Name: *ROTICH DERICK KEMBOI*

Registration Number: *INTE/MK/3212/09/24*

Course: IT

Activity 3: Assignment

Design and implement a simple client-server application on your localhost environment. Demonstrate the interaction between the client and server components, the layered architecture used, show successful communication between the system components and provide a link for the assessment

Layered architecture

Client application(client.py)



TCP socket layer



Localhost network



TCP socket layer



Server application(server.py)

Explanation of Layers:

Application Layer: The client and server programs written in Python.

Transport Layer: TCP protocol provides reliable communication.

Network Layer: Communication occurs through localhost (127.0.0.1).

Physical Layer: Simulated internally by the operating system because both programs run on the same computer.

Implementation

Implementation Procedure

1. Created server.py to listen for incoming client connections.
2. Created client.py to connect to the server.
3. Configured both programs to use IP address 127.0.0.1 and port 5000.
4. Started the server application first.
5. Started the client application.
6. Sent a test message from the client to the server.
7. Received a response from the server.

Successful communication

The server was started first, followed by the client. The client sent the message 'Hello server' and the server replied 'Server received: Hello server', confirming successful communication.

Screenshots of the server and client terminal

The screenshot displays the Visual Studio Code interface with a project named 'Clientserverapp'. The Explorer sidebar on the left shows the file structure with 'server.py' and 'client.py' under the 'CLIENTSERVERAPP' folder. The main editor window shows the code for 'server.py'.

```
1 import socket
2
3 HOST = "127.0.0.1"
4 PORT = 5000
5
6 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7 server.bind((HOST, PORT))
8 server.listen(1)
9
10 print("Server is running...")
11 print("Waiting for client...")
12
13 conn, addr = server.accept()
14 print("Connected by:", addr)
15
16 while True:
17     data = conn.recv(1024)
18     if not data:
19         break
20
21 message = data.decode()
22 print("Client:" + message)
```

The bottom panel shows the TERMINAL output, which matches the code's execution:

```
PS C:\xampp\htdocs\Clientserverapp> & 'C:\Python312\python.exe' 'c:\Users\DERICK ROTICHA\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '61578' '--' 'c:\xampp\htdocs\Clientserverapp\server.py'
Server is running...
Waiting for client...
Connected by: ('127.0.0.1', 61585)
Client: Hello server
PS C:\xampp\htdocs\Clientserverapp>
```

The status bar at the bottom indicates the current line is 19, column 14, with 4 spaces, UTF-8 encoding, CRLF line endings, Python language, and Python 3.12 (64-bit) interpreter.

```
1 import socket
2
3 HOST = "127.0.0.1"
4 PORT = 5000
5
6 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7 server.bind((HOST, PORT))
8 server.listen(1)
9
10 print("Server is running...")
11 print("Waiting for client...")
12
13 conn, addr = server.accept()
14 print("Connected by:", addr)
15
16 while True:
17     data = conn.recv(1024)
18     if not data:
19         break
20
21     message = data.decode()
22     print("Client:" + message)
```

```
PS C:\xampp\htdocs\Clientserverapp> python client.py
Enter message: Hello server
Server received: Hello server
PS C:\xampp\htdocs\Clientserverapp>
```

Conclusion

The application successfully demonstrated reliable client-server communication using TCP sockets over localhost.